

DATABASE ADMINISTRATION

Alexandru Ioan Cuza University | Master's Programme | 2025-2026

LAB 3 — SQL Processing & Query Optimization

Duration	90 minutes
Oracle	Oracle 23ai / 19c / 12c (SQL Developer)
Prerequisites	Week 3 lecture completed
Assessment	Moodle Flash-Quiz (due 24h after lab)

Learning Objectives

- Measure hard vs soft parse ratios using V\$SYSSTAT
- Generate estimated plans (EXPLAIN PLAN) and actual plans (GATHER_PLAN_STATISTICS)
- Read ALLSTATS LAST output — interpret E-Rows vs A-Rows discrepancy
- Understand why SIZE 1 stats force the 1/NDV formula and why histograms fix it
- Create and verify Function-Based, covering, and invisible indexes
- Apply the 5-step tuning workflow to a slow query

Schema Setup — Run Before Starting

The setup script creates three tables (10K customers, 500K orders with skewed STATUS, 1.5M items), inserts data in batches of 100K to avoid ORA-30009 PGA memory errors, and gathers basic statistics with SIZE 1 — meaning Oracle knows NDV=5 for STATUS but has no histogram. This forces the CBO to use the uniform SEL = 1/NDV formula.

Why SIZE 1 instead of no stats?

If no statistics exist at all, Oracle triggers Dynamic Sampling — it reads a sample of actual data blocks during parsing and gets a reasonably accurate estimate (~25K for 'OPEN'). This hides the 1/NDV problem entirely. By gathering stats with SIZE 1, Oracle has NDV=5 but believes all 5 values appear equally (20% each). Exercise 2 depends on this misconception to demonstrate the cardinality error.

```
-- Run as LAB_USER - setup script
CREATE TABLE customers (id NUMBER PRIMARY KEY, name VARCHAR2(100),
    region VARCHAR2(5), created_dt DATE);
CREATE TABLE orders (id NUMBER PRIMARY KEY,
    customer_id NUMBER REFERENCES customers(id),
    status VARCHAR2(10), amount NUMBER(12,2), order_dt DATE);
CREATE TABLE order_items (id NUMBER PRIMARY KEY,
    order_id NUMBER REFERENCES orders(id),
    product VARCHAR2(50), qty NUMBER, price NUMBER(10,2));

-- Customers (10K - single batch)
INSERT INTO customers
```

```
SELECT level, 'Customer '||level,
       CASE MOD(level,4) WHEN 0 THEN 'EU' WHEN 1 THEN 'US'
              WHEN 2 THEN 'ASIA' ELSE 'ROW' END,
       SYSDATE - DBMS_RANDOM.VALUE(1,1000)
FROM dual CONNECT BY level <= 10000;
COMMIT;

-- Orders (5 batches x 100K – avoids ORA-30009)
BEGIN
  FOR batch IN 0..4 LOOP
    INSERT INTO orders
      SELECT batch*100000+level,
             TRUNC(DBMS_RANDOM.VALUE(1,10001)),
             CASE WHEN MOD(batch*100000+level,20)<17 THEN 'CLOSED'
                   WHEN MOD(batch*100000+level,20)<18 THEN 'OPEN'
                   WHEN MOD(batch*100000+level,20)<19 THEN 'PENDING'
                   WHEN MOD(batch*100000+level,20)<20 THEN 'CANCELLED'
                   ELSE 'REFUNDED' END,
             ROUND(DBMS_RANDOM.VALUE(10,5000),2),
             SYSDATE - DBMS_RANDOM.VALUE(1,730)
      FROM dual CONNECT BY level <= 100000;
    COMMIT;
  END LOOP;
END;
/

-- Order Items (15 batches x 100K – avoids ORA-30009)
BEGIN
  FOR batch IN 0..14 LOOP
    INSERT INTO order_items
      SELECT batch*100000+level,
             MOD(batch*100000+level,500000)+1,
             'Product '||MOD(batch*100000+level,200),
             TRUNC(DBMS_RANDOM.VALUE(1,10)),
             ROUND(DBMS_RANDOM.VALUE(5,500),2)
      FROM dual CONNECT BY level <= 100000;
    COMMIT;
  END LOOP;
END;
/

-- Gather stats with SIZE 1 (no histograms – forces 1/NDV)
BEGIN
  DBMS_STATS.GATHER_SCHEMA_STATS(
    ownname => USER,
    estimate_percent => DBMS_STATS.AUTO_SAMPLE_SIZE,
    method_opt => 'FOR ALL COLUMNS SIZE 1',
    cascade => TRUE,
    no_invalidate => FALSE);
END;
/
```

DONE!

Exercises

Ex 1 Hard vs Soft Parse

15 min

Prove that literal SQL causes hard parses and bind variables eliminate them.

Step 1 — Record baseline parse counts

```
SELECT name, value FROM v$sysstat
WHERE name IN ('parse count (total)', 'parse count (hard)');
```

Baseline values:

NAME	VALUE

parse count (total)	14889
parse count (hard)	4502

Step 2 — 100 hard parses (literal SQL)

```
BEGIN
  FOR i IN 1..100 LOOP
    EXECUTE IMMEDIATE 'SELECT COUNT(*) FROM orders WHERE id = '||i;
  END LOOP;
END;
/
```

DONE

Step 3 — Measure the increase

```
SELECT name, value FROM v$sysstat
WHERE name IN ('parse count (total)', 'parse count (hard)');
```

Hard parses increased by:

NAME	VALUE

parse count (total)	15070
parse count (hard)	4634

Step 4 — Repeat with bind variables

```
DECLARE v NUMBER;
BEGIN
  FOR i IN 1..100 LOOP
    SELECT COUNT(*) INTO v FROM orders WHERE id = i;
  END LOOP;
END;
/
```

Hard parse increase this time:

NAME	VALUE

parse count (total)	15194
parse count (hard)	4667

Discussion

What happens if you set `cursor_sharing = FORCE`? Is it a good long-term fix? What are the drawbacks for queries where different literal values need different execution plans?

Despre ce face `cursor_sharing = FORCE`:

Dacă activăm parametrul ăsta, Oracle ia toate valorile literale din codul SQL și le înlocuiește automat în fundal cu variabile de legătură.

Despre fix-ul pe termen lung:

Nu e deloc o soluție pe termen lung, e doar o cârpeală de moment. Fix-ul real este rescrierea codului din aplicație ca să folosească `bind variables` din start.

Despre dezavantajele pe planurile de execuție:

Marele dezavantaj e că ne strică performanța când datele sunt distribuite inegal. Ascunzând valoarea reală, Oracle blochează în memorie un singur plan de execuție pentru toate cazurile, așa că o căutare care ar fi fost foarte rapidă cu un Index ar putea fi forțată să scaneze tot tabelul.

Ex 2 Estimated vs Actual Execution Plans

20 min

Statistics exist with SIZE 1 (no histograms). Oracle uses $SEL = 1/NDV = 1/5 = 20\%$ and estimates 100,000 rows for 'OPEN'. The actual count is ~25,000. Your goal is to see this 4× gap.

Step 1 — Estimated plan (query is NOT executed)

```
EXPLAIN PLAN FOR
  SELECT c.name, COUNT(*) n_orders, SUM(o.amount) total
  FROM   customers c
  JOIN   orders o ON c.id = o.customer_id
  WHERE  o.status = 'OPEN'
  GROUP BY c.name;

SELECT * FROM TABLE(DBMS_XPLAN.display('PLAN_TABLE', NULL, 'TYPICAL'));
```

E-Rows at the ORDERS TABLE ACCESS step: (expected: ~100,000)

125K

Join method (HASH / NESTED LOOP / SORT-MERGE):

HASH JOIN

Does the Note section say 'dynamic statistics used'? (should be NO)

No

Step 2 — Actual plan (query IS executed)

```
SELECT /*+ GATHER_PLAN_STATISTICS */
  c.name, COUNT(*) n_orders, SUM(o.amount) total
  FROM   customers c
  JOIN   orders o ON c.id = o.customer_id
  WHERE  o.status = 'OPEN'
  GROUP BY c.name;

SELECT * FROM TABLE(
  DBMS_XPLAN.display_cursor(NULL, NULL, 'ALLSTATS LAST'));
```

A-Rows at the ORDERS step: (expected: ~25,000)

25000

E-Rows was _____, A-Rows was _____, ratio = _____× off

E-Rows was 125K, A-Rows was 25000, ratio = 5× off

What column needs a histogram to fix this?

STATUS

Key Insight

The CBO formula $SEL = 1/NDV = 1/5 = 0.20$ assumes every STATUS value has equal frequency. In reality OPEN = 5% and CLOSED = 95%. The 4× overestimate causes Oracle to choose Hash Join where Nested Loop + Index would be faster.

Ex 3 Statistics & Histograms

20 min

Add a histogram on the STATUS column. Observe how E-Rows changes from ~100,000 to ~25,000 for 'OPEN' (and from ~100,000 to ~425,000 for 'CLOSED').

Step 3a — Verify current state: stats exist, no histogram

```
SELECT column_name, histogram, num_distinct, num_buckets
FROM   user_tab_col_statistics
WHERE  table_name = 'ORDERS' AND column_name = 'STATUS';
```

HISTOGRAM value: (expected: NONE)

Funcția MOD(numar, 20) returnează restul împărțirii unui număr la 20.

Deoarece rezultatul funcției MOD nu poate fi niciodată 20 sau mai mare, ramura ELSE 'REFUNDED' nu este executată

NUM_DISTINCT value: (expected: 5)

HISTOGRAM	NUM_DISTINCT	NUM_BUCKETS
STATUS		
NONE	4	1

Step 3b — Gather histogram on STATUS

```
BEGIN
  DBMS_STATS.GATHER_TABLE_STATS(
    ownname => USER,
    tabname => 'ORDERS',
    method_opt => 'FOR COLUMNS STATUS SIZE AUTO');
END;
/

-- Verify histogram exists
SELECT column_name, histogram, num_distinct, num_buckets
FROM   user_tab_col_statistics
WHERE  table_name = 'ORDERS' AND column_name = 'STATUS';
```

HISTOGRAM value now: (expected: FREQUENCY)

HISTOGRAM	NUM_DISTINCT	NUM_BUCKETS
STATUS		
FREQUENCY	4	4

Step 3c — Verify cardinality for 'OPEN'

```
SELECT /*+ GATHER_PLAN_STATISTICS */ COUNT(*)
FROM   orders WHERE status = 'OPEN';

SELECT * FROM TABLE(
  DBMS_XPLAN.display_cursor(NULL, NULL, 'ALLSTATS LAST'));
```

E-Rows after histogram: (expected: ~25,000)

PLAN_TABLE_OUTPUT

```
-----
|* 2 | TABLE ACCESS FULL| ORDERS |    1 |    25000 | 25000 |00:00:00.01 |
2392 |
```

Step 3d — Also verify 'CLOSED' (the other direction)

```
SELECT /*+ GATHER_PLAN_STATISTICS */ COUNT(*)
FROM   orders WHERE status = 'CLOSED';

SELECT * FROM TABLE(
  DBMS_XPLAN.display_cursor(NULL, NULL, 'ALLSTATS LAST'));
```

E-Rows for CLOSED: (expected: ~425,000 — was 100,000 before histogram)

PLAN_TABLE_OUTPUT

```
-----
|* 2 | TABLE ACCESS FULL| ORDERS |    1 |    425K | 425K |00:00:00.01 |
2392 |
```

Ex 4 Index Design & Usage

20 min

Create three index patterns and verify each in the execution plan.

4a — Function-Based Index (FBI)

```
-- Run WITHOUT index first
SELECT /*+ GATHER_PLAN_STATISTICS */ id, name FROM customers
WHERE UPPER(name) = 'CUSTOMER 42';
SELECT * FROM TABLE(DBMS_XPLAN.display_cursor(NULL,NULL,'ALLSTATS LAST'));
```

Access path **WITHOUT** index:

TABLE ACCESS FULL

```
-- Create FBI and re-run
CREATE INDEX idx_cust_upper_name ON customers(UPPER(name));
-- Re-run the same SELECT and check plan
```

Access path **WITH** FBI:

INDEX RANGE SCAN (urmat de TABLE ACCESS BY INDEX ROWID)

Is it an **ACCESS** or **FILTER** predicate?

ACCESS

4b — Covering Index

```
SELECT /*+ GATHER_PLAN_STATISTICS */ o.id, o.status, o.amount
FROM orders o WHERE o.customer_id = 42;
SELECT * FROM TABLE(DBMS_XPLAN.display_cursor(NULL,NULL,'ALLSTATS LAST'));
```

Operations **BEFORE** covering index:

TABLE ACCESS FULL pe tabelul ORDERS, cu 2396 Buffers (logical reads)

```
CREATE INDEX idx_ord_cust_cover ON orders(customer_id, status, amount, id);
-- Re-run the same query
```

Operations **AFTER** covering index:

INDEX RANGE SCAN

Logical reads (Buffers) improvement:

A scăzut masiv, de la 2396 la doar 7.

4c — Invisible Index

```
CREATE INDEX idx_ord_status ON orders(status, customer_id) INVISIBLE;
ALTER SESSION SET OPTIMIZER_USE_INVISIBLE_INDEXES = TRUE;
SELECT /*+ GATHER_PLAN_STATISTICS */
* FROM orders WHERE status='OPEN' AND customer_id=42;
SELECT * FROM TABLE(DBMS_XPLAN.display_cursor(NULL,NULL,'ALLSTATS LAST'));
```

Does the invisible index improve the plan?

Da, planul arată clar că optimizatorul a ales să folosească indexul invizibil
IDX_ORD_STATUS printr-un INDEX RANGE SCAN.

Acest lucru confirmă că DBA-ul poate testa eficient un index nou direct în producție, fără ca
restul aplicațiilor să fie afectate, atâta timp cât indexul rămâne invizibil pentru ele.

Ex 5 Full Tuning Workflow — The Slow Query

15 min

Apply the 5-step tuning method to this query. Identify every anti-pattern and propose fixes.

The query to diagnose

```
SELECT /*+ GATHER_PLAN_STATISTICS */
       c.region, TO_CHAR(o.order_dt,'YYYY-MM') month,
       COUNT(*) orders, SUM(oi.qty * oi.price) revenue
FROM   customers c
JOIN   orders o      ON c.id = o.customer_id
JOIN   order_items oi ON o.id = oi.order_id
WHERE  UPPER(c.region) = 'EU'
AND    o.status <> 'CLOSED'
AND    o.order_dt >= SYSDATE - 180
GROUP BY c.region, TO_CHAR(o.order_dt,'YYYY-MM')
ORDER BY month;
```

```
SELECT * FROM TABLE(
  DBMS_XPLAN.display_cursor(NULL,NULL,'ALLSTATS LAST +PREDICATE'));
```

Elapsed time (A-Time at root):

00:00:00.18 (adică 0.18 secunde).

Logical reads (Buffers at root):

9667

Largest E-Rows vs A-Rows gap (which table?):

Tabelul CUSTOMERS

Identify anti-patterns (at least 2)

Utilizarea funcției UPPER() pe coloana region:

Aplicarea unei funcții direct pe o coloană în clauza WHERE "orbește" optimizatorul, strică estimările de cardinalitate (de aici gap-ul de la CUSTOMERS) și previne utilizarea oricărui index B-tree standard pe acea coloană.

Folosirea unei condiții de excludere / negative (AND o.status <> 'CLOSED'):

Operatorul de inegalitate <> este non-sargable (Search Argument Able). Asta înseamnă că baza de date nu poate folosi un index pentru a găsi eficient ce nu este acolo, fiind forțată să scaneze cantități uriașe de date inutile. O practică mult mai bună este transformarea ei într-o listă pozitivă de valori.

Hints

(1) UPPER(region) disables B-tree index. (2) status <> 'CLOSED' is non-sargable. (3) No index on order_dt for range predicate. (4) Consider rewriting <> to IN ('OPEN','PENDING','CANCELLED','REFUNDED').

Your proposed fix (SQL)

```

SELECT /*+ GATHER_PLAN_STATISTICS */
    c.region, TO_CHAR(o.order_dt,'YYYY-MM') month,
    COUNT(*) orders, SUM(oi.qty * oi.price) revenue
FROM   customers c
JOIN   orders o    ON c.id = o.customer_id
JOIN   order_items oi ON o.id = oi.order_id
WHERE  c.region = 'EU'
AND    o.status IN ('OPEN', 'PENDING', 'CANCELLED', 'REFUNDED')
AND    o.order_dt >= SYSDATE - 180
GROUP BY c.region, TO_CHAR(o.order_dt,'YYYY-MM')
ORDER BY month;

SELECT * FROM TABLE(
    DBMS_XPLAN.display_cursor(NULL,NULL,'ALLSTATS LAST +PREDICATE'));

```

Verify improvement

Elapsed time AFTER fix:

00:00:00.09

Logical reads AFTER fix:

9629

Improvement factor:

Time: A scăzut de la 0.18s la 0.09s, deci execuția este de 2 ori mai rapidă.

Moodle Flash-Quiz Preparation

1. What V\$SYSSTAT name measures hard parses?
2. Which DBMS_XPLAN format shows actual row counts from the last execution?
3. If E-Rows=100,000 and A-Rows=5,000, what is the most likely cause?
4. Why does GATHER_SCHEMA_STATS with SIZE 1 produce different CBO behaviour than having no stats at all?
5. Why does WHERE UPPER(col) = 'VALUE' prevent a regular B-tree index from being used?
6. What parameter allows you to test an INVISIBLE index in your session without affecting others?